26. FAST INFERENCE

31 March 2026 17:37

Inference from large auto regressive models like transformers is slow

decoding K-tokens takes K serial runs of the model

Speculative decoding An algorithm to sample from auto regressive models faster without any changes to the outputs by computing several tokens in parallel

① Hard language modeling tasks often include easier subtasks that can be approximated well by more efficient models

② Using speculative execution and a novel sampling method

We can make exact decoding from the large models faster by running them parallel on the outputs of approximation models, potentially generating several tokens concurrently and without changing the distribution

Same distribution
But Faster Inference

Smaller model generates candidate tokens

The system still respects autoregressive structure
But tries to amortize multiple tokens into fewer large model invocations

→ The smaller model runs ahead → The larger model later checks

Probabilities relate to sampling/acceptance behavior

Constraint
Autoregressive dependency → serialization

Objective
Reduce latency, Preserve exact distribution

System insight
Bottleneck = memory bandwidth, not FLOPs

→ Autoregressive bottleneck
K tokens → K serial runs
dependency chain blocks naive parallelism

→ Problem facing
Inference latency grows over sequence length
Scaling models → worse latency

→ High level idea
Use smaller/efficient model alongside large one
Introduce parallelism despite autoregressive constraint
goal same output distribution

→ Speculative execution Intuition
"do work before knowing it's needed"
parallel generation + later verification

→ Production constraint
no retraining/no architecture change

→ Memory bound insight
Inference bottleneck ≠ compute
but memory bandwidth

Speculative Decoding

M_p -> target model

$p(x_t | x_{<t})$

M_q → more efficient approximation model for the same task

$q_i(x_t | x_{<t})$

use M_q to generate $\gamma \in \mathbb{Z}^+$ completions

then use M_p to evaluate all of the guesses and their respective probabilities from M_q in parallel accepting all that can lead to an identical distribution

then use M_p to evaluate all of the guesses and their respective probabilities from M_q in parallel
accepting all that can lead to an identical distribution

then sampling an additional token from an adjusted distribution to fix the first one that
was rejected, or to add an additional token if they are all accepted

Though above each parallel run of M_p will produce at least one new token
(so the number of serial runs of the target model can never, even in the worst case.
be larger than the simple autoregressive method)

But it can potentially generate many new tokens, upto $\gamma+1$, depending on how well M_q approximates M_p

Standardized Sampling from an adjusted probability distribution

Assume $p(x)$ $q(x)$ as distributions from the M_p and M_q

Speculative sampling

To sample $x \sim p(x)$
we instead sample $x \sim q(x)$

Keep it if $q(x) \leq p(x)$

in case $q(x) > p(x)$ we reject the sample with probability $1 - \frac{p(x)}{q(x)}$

Sample again from from an adjusted distribution $p'(x) = \text{norm}\left(\max\left(0, p(x) - q(x)\right)\right)$

for any distributions $p(x)$ and $q(x)$ & x sampled in this way indeed $x \sim p(x)$

$q(x)$ obtained from M_q on a conditioning prefix,

Sample a token $x_1 \sim q(x)$

then calculate the dist'n $p(x)$ by running M_p on prefix

While in parallel speculatively calculating the distb'n of next token x_2,
by running M_p on prefix + $[x_1]$

Once both complete

if x_1 is rejected, we discard the computation of x_2 and resample x_1 from
the adjusted distribution

if x_1 is accepted, we keep both tokens

the algo generalizes this idea to sample between 1 and $\gamma+1$ tokens at once

Small model M_q

generates a sequence of guesses

$x_1, x_2, \ldots x_\gamma$ done autoregressively

Small model

proposes a trajectory (sequence of tokens)

Large model M_p

Large model

Large model M_p
evaluates $p(x_1)\ p(x_2)\ p(x_{t+1})$
done in parallel

Acceptance process
compare guesses x_i using acceptance condition involving p and q
Find first rejection index n'

Output construction
Accept tokens up to $n-1$
Then either adjust distribution or sample new token

Information flow
M_q generates $x_1 \ldots x_\gamma$ (small cheap)
M_p evaluates them (parallel, expensive)
accept/reject → construct output

Large model
Validates that trajectory probabilistically
evaluates all guesses at once (parallel)

Representation
$q(x_t | \text{prefix})$ small model distribution
$p(x_t | \text{prefix})$ large model distribution

Objective Sample from p
But reduce no of sequential calls to M_p

Memory behavior
Instead of
many small sequential memory accesses (bad)
We do
fewer wider parallel accesses (better utilization)

The probability each model provides is essentially a softmax over the vocabulary
Both models produces a vector of scores (logits) for all possible next tokens → then use softmax

Analysis
what is β?
$\beta_{x<t}$ is the probability of accepting $x_t \sim q(x_t | x_{<t})$ using speculative sampling
X not a token/distribution
for one specific prefix $x<t$ small model proposes a token
the probability that it gets accepted is $\beta_{x<t}$

$E(\beta)$ → a measure of how well M_q approximates M_p
↳ instead of considering millions of values of $\beta_{x<t}$
$\alpha = E(\beta)$
↙
Average acceptance probability
if M_q is very similar to M_p $\alpha \approx 1$
M_q is poor α becomes very smaller

They say no of tokens produced, is a capped geometric variable with success probability $1-\alpha$

They say no of tokens produced, is a capped geometric variable with success probability $1-\alpha$

here success : "token accepted"

but stopping event occurs (which is first rejection)

$\alpha \rightarrow$ acceptance probability

$1-\alpha \rightarrow$ rejection probability

$E(\#\text{ generated tokens}) = \frac{1-\alpha^{\gamma+1}}{1-\alpha}$ (Geometric progression)

They define natural Divergence $D_{LK}(P,q) = \sum_x |P(x) - M(x)| \quad M = \frac{P(x)+q(x)}{2}$

$= \sum_x |q(x) - M(x)|$

$D_{LK}(P,q) = 1 - \sum_x \min(P(x), q(x))$

Proof $D_{L.K}(P,q) = \sum_x \left| P(x) - \frac{P(x)+q(x)}{2} \right|$

$= \sum_x \left| \frac{P(x) - q(x)}{2} \right|$

$\rightarrow \frac{1}{2} \sum_x |P(x) - q(x)|$

for any two numbers a, b

$|a-b| = a+b-2\min(a,b)$

Verify if $a>b$ $\quad a+b-2b = a-b$

if $a<b$ $\quad a+b-2a = b-a$

$|P(x) - q(x)| = P(x)+q(x) - 2\min(P(x), q(x))$

$D_{LK}(P,q) = \frac{1}{2} \sum_x (P(x)+q(x) - 2\min(P(x), q(x))$

Since P & q are distb^{n}s

$\sum_x P(x) = 1 \quad \sum_x q(x) = 1$

$\Rightarrow D_{LK}(P,q) = \frac{1}{2}(1+1 - 2\sum_x \min(P(x), q(x)))$

$\Rightarrow 1 - \sum_x \min(P(x), q(x))$

This is the amount of probability mass that the two distributions "agree on"

"Overlap Mass"

Representation view: Imagine P & q as two histograms $\sum_x (p,q)$ is literally the area where these histograms overlap

If they are identical $p=q \Rightarrow \sum_x \min(p,q) = 1 \Rightarrow D_{LK} = 0$

If they have completely disjoint support $\sum_x \min(p,q) = 0 \Rightarrow D_{LK} = 0$

the form $1 - \sum_x \min(p,q)$ is easier to connect to

acceptance probabilities

overlap between M_p and M_q

Theorem 35 $\beta = 1 - D_{LK}(p,q)$

$$\beta = E_{x \sim q(x)} \begin{cases} 1 & q(x) \le p(x) \\ \frac{p(x)}{q(x)} & q(x) > p(x) \end{cases}$$

$$= E_{x \sim q(x)} \min\left(1, \frac{p(x)}{q(x)}\right) = \sum_x \min(p(x), q(x))$$

β was an acceptance probability
D_{LK} was a divergence

Theorem states
Acceptance probability = 1 - divergence

A token is accepted when $r \le \frac{p(x)}{q(x)}$ $r \sim U(0,1)$

(i) if $q(x) \le p(x)$ then $\frac{p(x)}{q(x)} \ge 1$ (ii) if $q(x) > p(x) \Rightarrow \frac{p(x)}{q(x)} < 1$

Since $r \in [0,1]$
the token is always accepted
acceptance probability = 1

Acceptance probability becomes
exactly $\frac{p(x)}{q(x)}$

thus $P(\text{acceptance} \mid x) = \min\left(1, \frac{p(x)}{q(x)}\right)$

$\Rightarrow$ Average over tokens drawn from q

$\beta = E_{x \sim q}\left[\min\left(1, \frac{p(x)}{q(x)}\right)\right]$ ← if $p > q$ $q1 = q$

$$\beta = E_{x\sim q}\left[\min\left(1, \frac{p(x)}{q(x)}\right)\right]$$

$$\beta = \sum_x q(x) \min\left(1, \frac{p(x)}{q(x)}\right)$$

$$\beta = \sum_x \min\left(p(x), q(x)\right)$$

if $p > q$ $q\,1 = q$

if $p < q$ $q\,\frac{p}{q} = p$

In both cases $\min(p, q)$

$$D_{LK} = 1 - \sum \min(p(x), q(x)) \quad \text{(from earlier)}$$

↓ rearrange

$$\beta = 1 - D_{LK}(p, q)$$

The theorem says the probability a speculative token is accepted equals the overlap mass between the two distributions

Corollary 36 $\alpha = 1 - E(D_{LK}) = E(\min(p, q))$

↳ At any decoding position $\beta_{x<l} = 1 - D_{LK}(p, q)$

$$\alpha = E(\beta)$$

$$\alpha = E\left(1 - D_{LK}(p, q)\right)$$

$$\alpha = 1 - E\left(D_{LK}(p, q)\right)$$

↳ $1 - \sum_x \min(p(x), q(x))$

$$\alpha = E\left[\sum_x \min(p(x), q(x))\right]$$

$$\alpha = E(\min(p, q))$$

↳ average overlap mass across decoding positions

↳ summarizes how often does the small model agree with the large model

↳ directly determines expected accepted guesses
expected generated tokens
speed up

c, cost coefficient, be the ratio between time for a single run of M_q and the time for a single run of M_p

Theorem 38 The expected improvement factor in total walltime by Algorithm suggested

$$\frac{1 - \alpha^{\gamma+1}}{}$$

$$\frac{1-\alpha^{\gamma+1}}{(1-\alpha)(\gamma c+1)}$$

$c = \frac{\text{time of one } M_q \text{ run}}{\text{time of one } M_p \text{ run}}$ let T = time for one run of M_p

Small model cost

γ speculative guesses

$Tc\gamma$

Large model cost T

for one invocation total $T(1+c\gamma)$ from earlier $E(\#\text{tokens}) = \frac{1-\alpha^{\gamma+1}}{1-\alpha}$

Cost per token $\frac{\text{time}}{\text{token}} = \frac{T(1+c\gamma)}{(1-\alpha^{\gamma+1})/(1-\alpha)} \Rightarrow \frac{T(1-\alpha)(1+c\gamma)}{1-\alpha^{\gamma+1}}$

Comparison with baseline

one token, one M_p call T per token

Speed up → baseline cost / new cost

$$= \frac{T}{T\frac{(1-\alpha)(1+c\gamma)}{1-\alpha^{\gamma+1}}}$$

$$\Rightarrow \frac{1-\alpha^{\gamma+1}}{(1-\alpha)(1+\gamma c)}$$

$1-\alpha^{\gamma+1}$ → How many speculative tokens survive

$(1-\alpha)(1+\gamma c)$ → cost of running M_q ($c\gamma$)

↓ rejection behavior

Limiting checks

if $c=0$, small model free speedup $= \frac{1-\alpha^{\gamma+1}}{1-\alpha}$

meaning if speculation is free, runtime improvement equals tokens generated per step

if $\alpha \approx 0$, almost everything rejected speedup $\approx \frac{1}{1+c\gamma}$ You only pay overhead

if $\alpha = 1$, small model nearly identical to large model, accepted guesses become abundant

speed grows significantly

speed grows significantly

γ = how aggressively do we speculate?

α = How often those speculations are right?

c = How expensive is speculation?

Corollary 39 if $\alpha > c$, there exists γ for which we'll get an improvement

the improvement factor will be atleast $(1+\alpha)/(1+c)$

If we get an improvement for γ we'd get improvement for any $0 < \gamma^* < \gamma$

$$\text{Speedup} = \frac{1-\alpha^{\gamma+1}}{(1-\alpha)(1+c\gamma)} \qquad \gamma = 1 \qquad \alpha > c$$

$$\Rightarrow \frac{1-\alpha^2}{(1-\alpha)(1+c)}$$

$$= \frac{1+\alpha}{1+c}$$

$\alpha > c$ means agreement rate exceeds the cost rate

$\hat{c}$ = ratio of arithmetic operations per token of the approximation model M_q to that of the target model M_p

Standard decoding

for one token

M_p one forward pass

$$\hat{c} = \frac{\text{FLOPs of } M_q}{\text{FLOPs of } M_p}$$

Speculative decoding

for one speculative step

M_p (pos_1)
M_p (pos_2)
M_p (pos_γ+1)

} all evaluated together

Concurrent arithmetic increases roughly by $\gamma+1$

Grained parallelism Parallelism is not the same thing as reducing total work

Theorem 311 → The expected factor of increase in the number of total operations of Algorithm 1

is $\frac{(1-\alpha)(\gamma\hat{c}+\gamma+1)}{(1-\alpha^{\gamma+1})}$

an increase factor here

an increase factor here

Value = 1 → same no of operations baseline
Value = 2 → twice as many operations
Value = 0.8 → fewer operations

$\hat{T}$ → arithmetic ops done by standard decoding per token

$\hat{T}\hat{c}\gamma + \hat{T}(\gamma+1)$ → cost of one speculative iteration

Small-model cost $\hat{T}\hat{c}\gamma$
Large-model cost $(\gamma+1)\hat{T}$

In standard decoding 1 iteration → 1 token

In speculative decoding 1 iteration → multiple tokens $\left(\frac{1-\alpha^{\gamma+1}}{1-\alpha}\right)$

So operations per generated token ⇒ $\frac{\hat{T}(\gamma\hat{c}+\gamma+1)}{(1-\alpha^{\gamma+1})/(1-\alpha)}$

$\Rightarrow \frac{\hat{T}(1-\alpha)(\gamma\hat{c}+\gamma+1)}{(1-\alpha^{\gamma+1})}$ → small model work + large model work

⇒ operations per iteration / tokens per iteration

Total no of memory accesses can go down with this method

Choosing γ → given α and c

Increase γ → speculate further (potentially more accepted tokens)
→ you pay $(1+c\gamma)$ → more cost)

→ Hence 'γ' an optimization variable

optimal
as α increases γ increases (it becomes worthwhile to speculate further)

If c increases optimal γ decreases (Speculation becomes more expensive)

α → Global average acceptance probability

β → Acceptance probability for a specific prefix

Authors observed β changing throughout generation (Some regions of text are easier while others harder)

↳ Hence a fixed 'γ' may not be optimal

if you could predict β at runtime, you could adapt γ dynamically → future work

Approximation Models

what to use for M_q → Almost anything since Alg. 1 guarantees output distribution

eg same-family → smaller models

Negligible cost models → $c \approx 0$ (n-gram models, repetition based methods)

Approximation models doesn't have to be a transformer

One limitation

latency is improved through increased concurrency at the cost of an increased no of arithmetic operations